

CodeMuscle v1 — Functional Specification

1. Product summary

CodeMuscle is a lightweight coding interview revision tracker.

It helps users:

- Build a personal coding problem library
- Record practice attempts
- Decide which problems to revise next
- Track retention across topics and patterns

The main product question is:

What coding problems should I revise today?

CodeMuscle should support users who already have preparation data in Excel or CSV, as well as users starting from scratch.

2. Product goals

CodeMuscle should:

1. Let users create a coding problem library.
 2. Preserve existing preparation history when importing data.
 3. Record every new practice attempt.
 4. Calculate the next revision date for each problem.
 5. Generate a daily revision queue.
 6. Show progress and weak areas by topic.
 7. Store personal data outside the public source repository.
 8. Remain simple enough for users to clone and run locally.
-

3. Non-goals

CodeMuscle v1 will not include:

- A code editor
- Code execution
- Automatic LeetCode login
- Automatic submission history import
- Browser automation

- Social features
 - Leaderboards
 - Job application tracking
 - Resume management
 - System design preparation
 - Behavioral interview preparation
 - Mobile applications
 - Multi-user collaboration
-

4. Core user flows

4.1 Create a problem library

A user can populate their library using one of these methods:

- Import Excel
- Import CSV
- Add problems manually
- Start with an empty library

All methods must create the same internal problem records.

Excel and CSV are import options, not the foundation of the product.

4.2 Record a practice attempt

The user selects a problem and records:

- Whether they solved it
- Whether they used hints
- Time spent
- Confidence
- Optional notes

CodeMuscle then updates:

- Attempt history
 - Mastery state
 - Revision count
 - Continuous successful revisions
 - Next revision date
-

4.3 Generate today's revision queue

The user provides available preparation time.

CodeMuscle recommends a list of problems based on:

- Problems due today
- Overdue problems
- Previously failed problems
- Weak topics
- Time since last revision
- Repeated successful revisions
- User priority

Each recommendation should explain why it was selected.

4.4 Review progress

The user can see:

- Problems due today
 - Problems overdue
 - Problems mastered
 - Problems needing relearning
 - Topic-level success rates
 - Topic-level revision activity
 - Recent practice history
-

5. Core concepts

5.1 Problem

A problem is one coding question in the user's library.

A problem should contain:

- Title
- URL
- Platform or source
- Difficulty
- Topics
- Patterns
- Notes

- Priority
- Date added
- Current mastery state
- Next revision date
- Archived status

Only the title should be required.

5.2 Topic

A topic is a broad data structure or algorithm area.

Examples:

- Arrays
- Strings
- Linked Lists
- Stacks
- Queues
- Heaps
- Trees
- Graphs
- BFS
- DFS
- Dynamic Programming
- Greedy
- Backtracking
- Binary Search
- Intervals

A problem may have multiple topics.

5.3 Pattern

A pattern is a reusable problem-solving technique.

Examples:

- Two Pointers
- Sliding Window
- Prefix Sum
- Topological Sort
- Fast and Slow Pointers
- Monotonic Stack
- Merge Intervals

- Binary Search on Answer

A problem may have multiple patterns.

Patterns are optional in v1.

5.4 Attempt

An attempt represents one practice session for one problem.

Each attempt should contain:

- Attempt date
- Outcome
- Hint usage
- Time spent
- Confidence
- Notes

Previous attempts must never be overwritten.

5.5 Mastery state

Each problem should have one current mastery state:

- New
- Learning
- Fragile
- Retained
- Mastered
- Needs Relearning
- Archived

The state should normally be calculated by CodeMuscle, but the user may override it.

6. Functional requirements

6.1 Workspace and privacy

CM-001: Create a private workspace

On first use, CodeMuscle should create or ask the user to select a private data directory.

This directory should contain:

- Problem data
- Attempt history
- Preferences
- Imports
- Exports
- Backups

The private workspace should not be stored inside the source repository by default.

CM-002: Separate source code and personal data

The public GitHub repository may contain:

- Application code
- Documentation
- Empty configuration templates
- Fictional sample data
- Tests

It must not contain:

- Real user problem history
- Personal notes
- API keys
- Imported spreadsheets
- User preferences
- Backups

CM-003: Support data export

The user should be able to export their data as:

- CSV
- JSON
- Excel, where supported

CM-004: Support backup and restore

The user should be able to:

- Create a backup
- Restore from a backup
- Delete all personal data

6.2 Problem library

CM-005: Add a problem manually

The user should be able to create a problem using:

- Title
- URL, optional
- Difficulty, optional
- Topics, optional
- Patterns, optional
- Notes, optional

CM-006: Edit a problem

The user should be able to edit all problem fields.

CM-007: Archive a problem

The user should be able to archive a problem without deleting its history.

CM-008: Search and filter problems

The library should support filtering by:

- Topic
- Pattern
- Difficulty
- Mastery state
- Due status
- Platform
- Archived status

CM-009: Detect duplicate problems

The system should warn about possible duplicates based on:

- URL
- Platform identifier
- Similar title

The user may still keep both records.

6.3 Import

CM-010: Import Excel and CSV

The user should be able to import `.xlsx` and `.csv` files.

CM-011: Map columns

Before import, the user should be able to map file columns to CodeMuscle fields.

Supported imported fields should include:

- Problem Link
- Problem Title
- Difficulty
- Notes
- Topic
- Pattern
- Solved First Time
- Last Revised or Solved Date
- Number of Revisions
- Number of Successful Continuous Revisions
- Next Revision Date

CM-012: Preview the import

The system should show:

- Total rows
- Valid rows
- Invalid rows
- Possible duplicates
- Proposed column mappings

CM-013: Preserve existing history

When fields are available, the importer should preserve:

- Previous revision count
- Continuous successful revisions
- Last revision date
- Notes
- Difficulty
- Topics

CM-014: Handle errors safely

Invalid rows should not block valid rows from being imported.

The user should be able to correct and retry failed rows.

6.4 Attempt tracking

CM-015: Record an attempt

The user should be able to record one of these outcomes:

- Solved independently
- Solved with a small hint
- Solved with significant help
- Understood after viewing the solution
- Failed
- Skipped

CM-016: Record supporting details

The user may also record:

- Time spent
- Confidence
- Notes
- Whether time and space complexity were understood

CM-017: Preserve attempt history

Every attempt should remain visible in chronological order.

CM-018: Update problem summary

After an attempt, the system should update:

- Last attempt date
 - Total attempts
 - Total revisions
 - Continuous successful revisions
 - Mastery state
 - Next revision date
-

6.5 Revision scheduling

CM-019: Calculate the next revision date

The system should calculate the next revision date using:

- Attempt outcome
- Hint usage
- Continuous successful revisions
- Difficulty
- Confidence
- User priority

CM-020: Use simple spaced repetition

The default successful revision intervals should be approximately:

- 1 day
- 3 days
- 7 days
- 14 days
- 30 days
- 60 days

A failed attempt should shorten the interval.

An independently solved attempt should increase the interval.

CM-021: Allow manual override

The user should be able to manually change the next revision date.

CM-022: Explain the schedule

The user should be able to see why the next revision date was selected.

Example:

Scheduled in 3 days because the problem was solved with a hint and has only one successful revision.

6.6 Daily revision queue

CM-023: Generate a daily queue

The user should be able to enter:

- Available time
- Optional topic focus
- Optional number of problems

The system should generate a queue.

CM-024: Prioritize relevant problems

The queue should prioritize:

1. Overdue problems
2. Failed problems
3. Fragile problems
4. Weak topics
5. Problems due today
6. High-priority problems

CM-025: Balance topics

The queue should avoid selecting only one topic unless the user explicitly requests it.

CM-026: Estimate session length

Each problem should have an estimated practice duration.

The queue should fit within the user's available time as closely as possible.

CM-027: Explain each recommendation

Each problem should include a simple reason.

Examples:

- Overdue by five days
- Previous attempt failed
- Heap is currently a weak topic
- Retention check after 30 days
- High-priority interview problem

CM-028: Allow queue changes

The user should be able to:

- Remove a problem
 - Replace a problem
 - Postpone a problem
 - Add another problem
 - Mark a problem complete
-

6.7 Statistics

CM-029: Show dashboard summary

The dashboard should show:

- Problems due today
- Problems overdue
- Problems practiced this week
- Problems mastered
- Problems needing relearning
- Current practice streak, optional

CM-030: Show topic statistics

For each topic, show:

- Total problems
- Attempts
- Independent success rate
- Problems due
- Problems overdue
- Mastery distribution
- Last practiced date

CM-031: Show problem history

For each problem, show:

- Full attempt history
- Revision dates
- Outcomes
- Notes
- Mastery changes

CM-032: Show weak topics

The system should identify topics that are:

- Weak
- Neglected
- Improving
- Stable

The system should explain how each status was determined.

7. Optional AI functionality

AI features must be optional.

CodeMuscle must remain usable without them.

7.1 Topic and pattern suggestions

AI may suggest topics and patterns based on:

- Problem title
- URL
- User notes
- Optional problem summary

The user must approve or edit suggestions.

7.2 Notes analysis

AI may identify common failure reasons such as:

- Forgot the pattern
- Incorrect edge-case handling
- Complexity misunderstanding
- Implementation errors
- Could not derive the recurrence
- Needed help with initialization

7.3 Weekly summary

AI may generate a summary such as:

You improved in trees this week, but heaps and graph traversal remain inconsistent. Prioritize three heap revisions and one BFS problem next week.

AI-generated insights must be based on stored user data.

8. Primary screens

CodeMuscle v1 should have these main screens.

8.1 Dashboard

Shows:

- Today's due problems
- Overdue problems
- Start revision session
- Recent activity
- Topic summary

8.2 Problem Library

Shows:

- Searchable problem list
- Filters
- Add problem
- Import data
- Edit problem
- Archive problem

8.3 Daily Queue

Shows:

- Available time
- Recommended problems
- Recommendation reason
- Queue controls
- Start or record attempt

8.4 Problem Details

Shows:

- Problem information
- Topics and patterns
- Current mastery
- Next revision date
- Attempt history
- Record attempt

8.5 Statistics

Shows:

- Topic statistics
- Pattern statistics
- Practice trends
- Weak and neglected areas

8.6 Settings and Data

Shows:

- Workspace location
 - Revision preferences
 - Import
 - Export
 - Backup
 - Restore
 - Delete data
 - Optional AI configuration
-

9. Main user stories

Problem library

As a user, I want to add coding problems manually so that I can use CodeMuscle without an existing spreadsheet.

As a user, I want to import my current Excel or CSV tracker so that I do not lose my existing history.

As a user, I want to organize problems by topic and pattern so that I can understand my preparation coverage.

Attempts

As a user, I want to record how I performed on each revision so that CodeMuscle can track my retention.

As a user, I want my full attempt history preserved so that I can see whether I am improving.

Scheduling

As a user, I want CodeMuscle to calculate the next revision date so that I do not need to manage dates manually.

As a user, I want to understand why a problem was scheduled so that I can trust the recommendation.

Daily queue

As a user, I want a revision queue that fits my available time so that I can start practicing without planning the session myself.

As a user, I want the queue to include weak and overdue areas so that I do not repeatedly practice only comfortable topics.

Statistics

As a user, I want topic-level statistics so that I can identify weak and neglected areas.

As a user, I want to see which problems I repeatedly forget so that I can focus on them.

Privacy

As a user, I want my personal preparation data stored outside the source repository so that I can safely use and contribute to the public project.

10. Minimum viable product acceptance criteria

CodeMuscle v1 is complete when a new user can:

1. Clone the public repository.
2. Create a private workspace.
3. Start with an empty problem library.
4. Add a problem manually.
5. Import problems from Excel or CSV.
6. Review and correct imported records.
7. Assign multiple topics to a problem.

8. Record an attempt.
 9. View complete attempt history.
 10. Receive a calculated next revision date.
 11. Manually override the revision date.
 12. Generate a daily queue based on available time.
 13. See why each problem was recommended.
 14. Modify the generated queue.
 15. View topic-level statistics.
 16. Identify overdue and weak problems.
 17. Export personal data.
 18. Back up and restore personal data.
 19. Delete all personal data.
 20. Confirm that personal information is not stored in the public repository.
-

11. Suggested implementation milestones

Milestone 1: Local workspace

- Private data directory
- Basic settings
- Problem data model
- Attempt data model
- Backup and export foundations

Milestone 2: Problem library

- Manual problem creation
- Edit and archive
- Topics and patterns
- Search and filters
- Duplicate warnings

Milestone 3: Import

- Excel import
- CSV import
- Column mapping
- Preview
- Error handling

Milestone 4: Attempt tracking

- Record attempt
- Attempt history
- Mastery calculation

- Revision counters

Milestone 5: Scheduling

- Revision algorithm
- Next revision dates
- Manual override
- Scheduling explanations

Milestone 6: Daily queue

- Time-bounded queue
- Topic balancing
- Recommendation reasons
- Queue adjustments

Milestone 7: Statistics

- Dashboard
- Topic statistics
- Problem history
- Weak and neglected topic detection

Milestone 8: Optional AI

- Topic suggestions
- Pattern suggestions
- Failure analysis
- Weekly summary

12. Product definition

CodeMuscle is a private, open-source coding interview revision tracker that helps users remember solved problems by scheduling revisions, generating daily practice queues, and measuring retention across coding topics.